S-shaped Ramps with initial Start Velocity

In order to use S-shaped ramps with initial start velocity, do as follows:

Action:

- Set *RAMPMODE*(1:0)=b'10 (register 0x20).
- Set S-shaped ramp type accordingly, as explained before.
- Set proper *VSTART* > 0 (register 0x25).
- Set *VSTOP* = 0 (register 0x26).

Result:
The S-shaped ramp starts with initial velocity.

PRINCIPLE:

→ *The initial acceleration value is equal to AMAX. The parameter ASTART is not considered. Consequently, ramp phase B1 is not performed.*

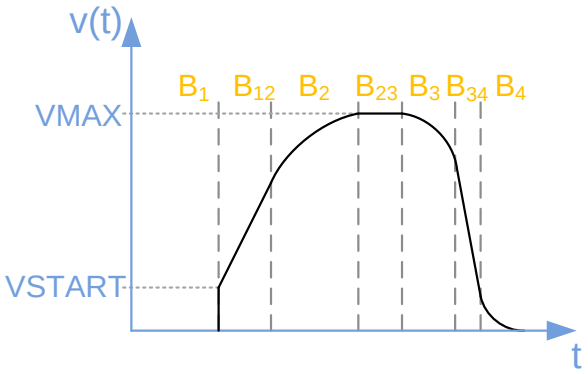


Figure 25: S-shaped Ramp with initial Start Velocity

If S-shaped ramp with initial velocity *VSTART* is selected:

NOTICE

Avoid unintended system behavior during positioning mode!

- Keep in mind that the S-shaped character of the curve is maintained. Because *AMAX* is the start acceleration value, the ramp will always execute phase B2 which could result in positioning overshoots.
- Use *VSTART* only in positioning mode if there is enough distance between the current position *XACTUAL* and the target position *XTARGET*.

This will ensure smooth operation during positioning mode.

•→Turn page for information on how to configure finishing ramps with stop velocity.

Finishing Ramps with Stop Velocity

S-shaped and trapezoidal velocity ramps can be finished with a stop velocity value if you set *VSTOP* value higher than zero (see figure below).

In order to configure trapezoidal ramps with stop velocity, do as follows:

Action:

- ➢ Set *RAMPMODE*(1:0)=b'01 (register 0x20).
- ➢ Set Trapezoidal ramp type accordingly, as explained before.
- ➢ Set *VSTART* = 0 (register 0x25).
- ➢ Set proper *VSTOP* > 0 (register 0x26).

Result:

The trapezoidal ramp stops with defined velocity.

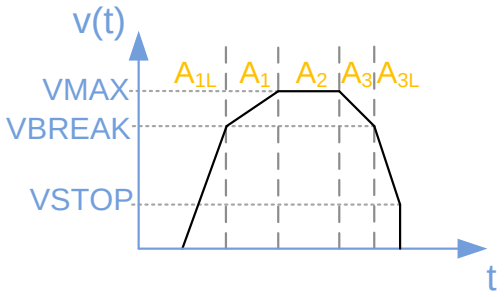


Figure 20: Trapezoidal Ramp with Stop Velocity

If trapezoidal ramps are selected (*VBREAK* > 0):

NOTICE

Avoid unintended system behavior during positioning mode!

- Set *VBREAK* > *VSTOP*.
- Set *VSTART* < *VSTOP*.

This will ensure smooth operation during positioning mode.

•→*Turn page for configuration information on S-shaped ramps with stop velocity.*

S-shaped Ramps with Stop Velocity

In order to use S-shaped ramps with stop velocity, do as follows:

Action:

- Set *RAMPMODE*(1:0)=b'10 (register 0x20).
- Set S-shaped ramp type accordingly, as explained before.
- Set *VSTART* = 0 (register 0x25).
- Set proper *VSTOP* > 0 (register 0x26).

Result:
The S-shaped ramp finishes with stop velocity.

NOTE:

→ *The final deceleration value is equal to DMAX. The parameter DFINAL is not considered. Consequently, ramp phase B4 is not performed.*

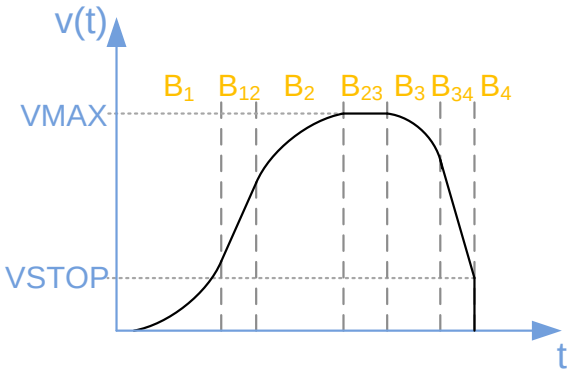


Figure 26: S-shaped Ramp with Stop Velocity

Interaction of *VSTART*, *VSTOP*, *VACTUAL* and *VMAX*:

- *VSTART* and *VSTOP* are only used to start or end a velocity ramp. If the velocity direction alters due to register assignments while a velocity ramp is in progress, the velocity values develop according to the current velocity ramp type without using *VSTART* or *VSTOP*.
- *VSTOP* can be used in positioning mode, if the target position is reached. In velocity mode, *VSTOP* can only be used if *VACTUAL* ≠ 0 and the target velocity *VMAX* is assigned to 0.
- The unsigned values *VSTART* and *VSTOP* are valid for both velocity directions.
- Every register value change is assigned immediately.

•→Turn page for information on how to configure S-shaped ramps with start and stop velocity.

6.4.1. S-shaped Ramps with Start and Stop Velocity

S-shaped ramps can be configured with a combination of *VSTART* and *VSTOP*.
It is possible to include both processes in one S-Shaped ramp to decrease the time between start and stop of the ramp.

In order to use S-Shaped ramps with a combination of start and stop velocity, do as follows:

Action:

- Set *RAMPMODE*(1:0)=b'10.
- Set S-shaped ramp type accordingly, as explained before.
- Set proper *VSTART* > 0 (register 0x25).
- Set proper *VSTOP* > 0 (register 0x26).

Result:
The S-shaped ramp starts with initial velocity and stops with defined velocity.

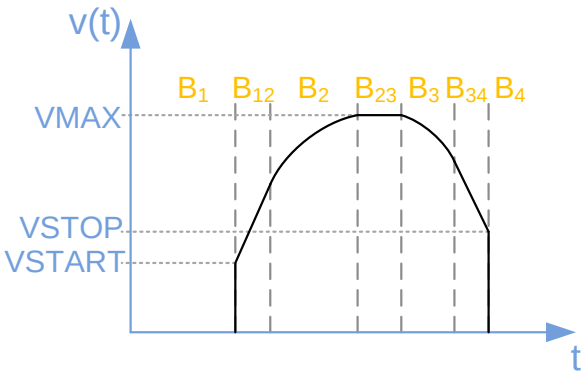


Figure 27: S-shaped Ramp with Start and Stop Velocity

If S-shaped ramp with initial velocity *VSTART* and stop velocity *VSTOP* is selected:

NOTICE

Avoid unintended system behavior during positioning mode!

- Keep in mind that the S-shaped character of the curve is maintained. Because *AMAX* is the start acceleration value, the ramp will always execute phase B2, which could result in positioning overshoots.
- Use *VSTART* in positioning mode, if there is enough distance between the current position *XACTUAL* and the target position *XTARGET*.

This will ensure smooth operation during positioning mode.

•→Turn page for information on how to use VSTART and ASTART for S-shaped ramps.

6.4.2. Combined use of *VSTART* and *ASTART* for S-shaped Ramps

For some S-shaped ramp applications it can be useful to start with a defined velocity value (*VSTART* > 0);but not with the maximum acceleration value *AMAX*.

In order to start with a defined velocity value, do as follows:

Action:

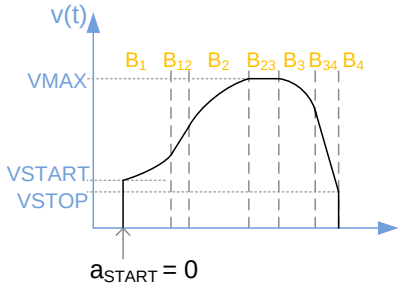
- Set *RAMPMODE*(1:0) =b'10 (register 0x20).
- Set S-shaped ramp type accordingly, as explained before.
- Set proper *VSTART* > 0 (register 0x25).
- Set proper *VSTOP* > 0 (register 0x26).
- Set *use_astart_and_vstart* =1 (bit0 of the *GENERAL_CONF* register 0x00).

Result:

The following special ramp types can be generated in this way, as shown below.

`i` Section B1 is passed through although *VSTART* is used.

Using VSTART *and starting acceleration of 0 for S-shaped ramps*



Using VSTART *and starting acceleration, which is smaller than* AMAX *for S-shaped ramps*

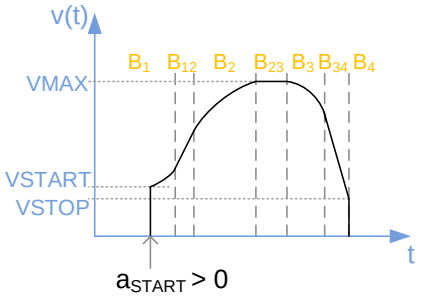


Figure 28: S-shaped Ramps with combined VSTART *and* ASTART *Parameters*

If S-shaped ramp with *VSTART*, *ASTART*, and *VSTOP* is selected:

NOTICE

Avoid unintended system behavior during positioning mode!

- Keep in mind that the S-shaped character of the curve is maintained. Because *ASTART* is the start acceleration value, the ramp will always execute phase B2, which could result in positioning overshoots.
- Use *VSTART* and *ASTART* > 0 without setting *VSTOP* > *VSTART* only in positioning mode, if there is enough distance between the current position *XACTUAL* and the target position *XTARGET*.

This will ensure smooth operation during positioning mode.

6.5. sixPoint Ramps

sixPoint ramps are trapezoidal ramps with initial and stop velocity values that also make use of two acceleration and two deceleration values.

Configuration of sixPoint Ramps

sixPoint ramps are trapezoidal velocity ramps that can be configured with a combination of *VSTART* and *VSTOP*.

In order to use trapezoidal ramps with a combination of start and stop velocity, do as follows:

Action:

- Set *RAMPMODE*(1:0)=b'01 (register 0x20).
- Set a Trapezoidal ramp type appropriately as explained before.
- Set proper *VSTART* > 0 (register 0x25).
- Set proper *VSTOP* > 0 (register 0x26).
- Set proper *VBREAK* > 0 (register 0x27).

Result:
The sixPoint ramp starts with an initial velocity and stops with a defined velocity.

Diagram of sixPoint Ramp

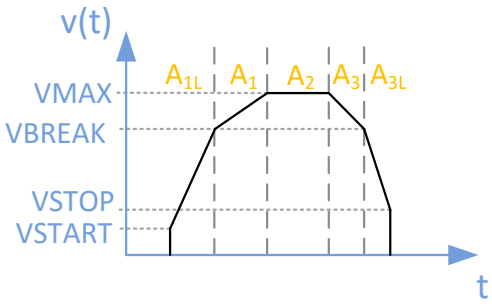


Figure 29: sixPoint Ramp: Trapezoidal Ramp with Start and Stop Velocity

If a sixPoint ramp is used:

NOTICE

Avoid unintended system behavior during positioning mode!

- Set *VBREAK* > *VSTOP*.
- Set *VSTART* < *VSTOP*.

This will ensure smooth operation during positioning mode.

Read entire documentation; especially the "Supplemental Directives" on page 218.

6.6. Internal Ramp Generator Units

This section provides information about the arithmetical units of the ramp parameters.

6.6.1. Clock Frequency

All parameter units are real arithmetical units.
Therefore, it is necessary to set the *CLK_FREQ* register 0x31 to proper [Hz] value, which is defined by the external clock frequency f_{CLK}. Any value between f_{CLK} = 4.2 MHz and 32 MHz can be selected.
Default configuration is 16 MHz.

6.6.2. Velocity Value Units

Velocity values are always defined as pulses per second [pps].
VACTUAL is given as a 32-bit signed value with no decimal places. The unsigned velocity values *VSTART*, *VSTOP*, and *VBREAK* consist of 23 digits and 8 decimal places. *VMAX* is a signed value with 24 digits and 8 decimal places.

The maximum velocity *VMAX* is restricted as follows:

$$VMAX \leq ½ \text{ pulse} \cdot f_{CLK}$$

NOTE:

→ *In case VACTUAL exceeds this limit INCORRECT step pulses at STPOUT output occur.*

6.6.3. Acceleration Value Units

The unsigned values *AMAX*, *DMAX*, *ASTART*, *DFINAL,* and *DSTOP* consist of 22 digits and 2 decimal places.
AACTUAL shows a 32-bit nondecimal signed value. Acceleration and deceleration units are defined per default as pulses per second² [pps²].

If higher acceleration/deceleration values are required for short and steep ramps, do as follows:

Action:
➢ Set *direct_acc_val_en* =1 (*GENERAL_CONF* register 0x00).

Result:
The parameters are defined as velocity value change per clock cycle with 24-bit unsigned decimal places (MSB =2^{-14}). The values are calculated as follows:

$$AMAX\ [pps^2] = AMAX / 2^{37} \cdot f_{CLK}^2$$
$$DMAX\ [pps^2] = DMAX / 2^{37} \cdot f_{CLK}^2$$
$$ASTART\ [pps^2] = ASTART / 2^{37} \cdot f_{CLK}^2$$
$$DFINAL\ [pps^2] = DFINAL / 2^{37} \cdot f_{CLK}^2$$
$$DSTOP\ [pps^2] = DSTOP / 2^{37} \cdot f_{CLK}^2$$

The maximum acceleration or deceleration values, in case *direct_acc_val_en* is activated, are as follows:

AMAX	≤ 65535
DMAX	≤ 65535
ASTART	≤ 65535
DFINAL	≤ 65535
DSTOP	≤ 65535

•→ *Continued on next page.*

6.6.4. Bow Value Units

Bow values BOW1...BOW4:
Bow values are unsigned 24-bit values without decimal places. They are defined per default as pulses per second³ [pps³].

In case higher bow values are required for short and steep ramps, do as follows:

Action:
➢ Set *direct_bow_val_en* =1 (*GENERAL_CONF* register 0x00)

Result:
The parameters are defined as acceleration value change per clock cycle with 24-bit unsigned decimal places with the MSB defined as 2^{-29}.

The particular bow values *BOW1*, *BOW2*, *BOW3*, *BOW4* are calculated as follows:

$$BOWx\ [pps^3] = BOWx\ /\ 2^{53} \cdot f_{CLK}^3$$

The maximum bow values, in case *direct_acc_val_en* is activated, are as follows:

$$BOW1...4 \leq 4095$$

6.6.5. Overview of Minimum and Maximum Values:

Minimum and Maximum Values (Frequency Mode)				
Value Classes	**Velocity**	**Acceleration**	**Bow**	**Clock**
Affected Registers	*VMAX, VSTART, VSTOP, VBREAK*	*AMAX, DMAX, ASTART, DFINAL*	*BOW1, BOW2, BOW3, BOW4*	*CLK_FREQ* (f_{CLK})
Minimum value	3.906 mpps	0.25 $mpps^2$	1 $mpps^3$	4.194 MHz
Maximum value	8.388 Mpps and ½ pulse · f_{CLK}	4.194 $Mpps^2$	16.777 $Mpps^3$	32 MHz

Table 18: Minimum and Maximum Values if Real World Units are selected

Minimum and Maximum Values for Steep Slopes (Direct Mode, example with f_{CLK} =16MHz)		
Value Classes	**Acceleration (*direct_acc_val_en* =1)**	**Bow (*direct_bow_val_en* =1)**
Affected Registers	*AMAX, DMAX, ASTART, DFINAL, DSTOP*	*BOW1, BOW2, BOW3, BOW4*
Value restrictions:	*< 65536*	*< 4096*
Calculation	$a[pps^2] = (\Delta v/clk\_cycle) / 2^{37} \cdot f_{CLK}^2$	$bow[pps^3] = (\Delta a/clk\_cycle) / 2^{53} \cdot f_{CLK}^3$
Minimum value	~1.86 $kpps^2$	~454.75 $kpps^3$
Maximum value	~122.07 $Mpps^2$	~1.86 $Gpps^3$

Table 19: Minimum and Maximum Values for Steep Slopes for f_{CLK} =16MHz

7. External Step Control and Electronic Gearing

Steps can also be generated by external steps that are manipulated internally by an electronic gearing process. In the following chapter, steps generation by external control and electronic gearing is presented.

Pins for External Step Control		
Pin Names	**Type**	**Remarks**
STPIN	Input	Step input signal.
DIRIN	Input	Direction input signal.

Table 20: Pins used for External Step Control

Registers used for external Step Control			
Register Name	**Register Address**		**Remarks**
GENERAL_CONF	0x00	RW	Bits 9:6, 26.
GEAR_RATIO	0x12	RW	Electronic gearing factor; signed; 32 bits=8+24 (8 bits integer part, 24 bits decimal places).

Table 21: Registers used for External Step Control

Enabling External Step Control

In order to synchronize with other motion controllers, TMC4361 offers a step direction input interface at the STPIN and DIRIN input pins.

i Three options are available. In case one of these options is selected, the internal step generator is disabled.

OPTION 1: HIGH ACTIVE EXTERNAL STEPS

Action:

➢ Set *sdin_mode* = b'01 (*GENERAL_CONF* register 0x00).

Result:

As soon as the STPIN input signal switches to high state the control unit recognizes an external step.

OPTION 2: LOW ACTIVE EXTERNAL STEPS

Action:

➢ Set *sdin_mode* = b'10 (*GENERAL_CONF* register 0x00).

Result:

As soon as the STPIN input signal switches to low state the control unit recognizes an external step.

OPTION 3: TOGGLING EXTERNAL STEPS

Action:

➢ Set *sdin_mode* = b'11 (*GENERAL_CONF* register 0x00).

Result:

As soon as the STPIN input signal switches to low or high state the control unit recognizes an external step.

•→ Continued on next page.

Selecting the Input Direction Polarity

DIRIN polarity can be assigned. Per default, the negative direction is indicated by DIRIN = 0.

In order to change this polarity:

Action:

➢ Set *pol_dir_in* = 1 (*GENERAL_CONF* register 0x00).

Result:

A negative input direction is assigned by DIRIN = 1.

7.1. Description of Electronic Gearing

If an external step is not congruent with an internal step, the *GEAR_RATIO* register 0x12 must be set accordingly. This signed parameter consists of eight bit digits and 24 bits decimal places.

With every external step the assigned *GEAR_RATIO* value is added to an internal accumulation register. As soon as an overflow occurs, an internal step is generated and the remainder will be kept for the next external step.

Any absolute gearing value between 2^{-24} and 127 is possible.

NOTE:

→ *Gearing ratios beyond 1 are more reasonable for the SPI output. The internal SinLUTable is used that generates multiple steps one after another without interpolation, if the accumulation register value is above 1. In contrast to a burst of steps at the STPOUT pin, the SPI output will only forward the new position in the inner SinLUT where only some values have been skipped if |GEAR_RATIO|>1.*

→ *A negative gearing factor* GEAR_RATIO < *0 inverts the interpretation of the input direction which is determined by DIRIN and* pol_dir_in*.*

7.2. Indirect External Control

It is possible to use the internal ramp generator in combination with the external S/D interface.

In this case, the external step impulses transferred via STPIN and DIRIN cannot influence the internal *XACTUAL* counter directly. Instead, the *XTARGET* register is altered by 1 or −1 with every *GEAR_RATIO* accumulation register overflow.

NOTE:

→ *Whether XTARGET is increased or decreased is determined similarly to the direct electronic gearing control. The accumulation register overflow direction indicates the target alteration. Respectively, the accumulation direction is determined by the GEAR_RATIO sign, by pol_dir_in, and by DIRIN.*

i This feature allows a synchronized motion of different positioning ramps for different TMC4361 chips with differently configured ramps.

In order to select indirect external control, do as follows:

Action:

➢ Set *sdin_mode* ≠ b'00 according to the required external control option.

➢ Set *sd_indirect_control* = 1 (*GENERAL_CONF* register 0x00).

Result:

As soon as an external step is generated, *XTARGET* is increased or decreased, according to the accumulation direction.

7.3. Switching from External to Internal Control

In some cases, it is useful to switch from external to internal ramp generation during motion.

TMC4361 supports a smooth transfer from direct external control to an internal ramp. The only parameter you need to know and apply is the current velocity when the switching occurs. In more detail, this means that when the external control is switched off, *VSTART* takes over the definition of the actual velocity value. The ramp direction is then selected automatically. The time step of the last internal step is also taken into account in order to provide a smooth transition from external to internal ramp control.

In order to select automatic switching from external to internal control, do as follows:

PRECONDITION (EXTERNAL DIRECT CONTROL IS ACTIVE):

Action:

- Set *sdin_mode* ≠ b'00 (*GENERAL_CONF* register 0x00).
- Set *sd_indirect_control* = 0 (*GENERAL_CONF* register 0x00).
- Set *ASTART* = 0 (register 0x2A).

PROCEED WITH:

Action:

- Set *automatic_direct_sdin_switch_off* = 1 (*GENERAL_CONF* register 0x00) once before switching to internal control.
- Continually adapt *VSTART* register 0x25 according to the actual velocity of the TMC4361 that must be calculated in the µC.
- If switching must be prompted, set *sdin_mode* = b'00.

Result:

The internal ramp velocity is started with the value of *VSTART*, and the direction is set automatically on the basis of the external steps that have occurred before.

•→Turn page for information on how configure smooth switching for S-shaped ramps.

Smooth Switching for S-shaped Ramps

In order to also support a smooth S-shaped ramp transition - when the external step control is switched off - the starting acceleration value can also be set separately at *ASTART* register 0x2A.

i In contrast to the automatic direction assignment, the sign of *ASTART* must be set manually.

In order to select automatic switching from external to internal control with a starting acceleration value, do as follows:

PRECONDITION (EXTERNAL DIRECT CONTROL IS ACTIVE):

Action:

- Set *sdin_mode* ≠ b'00 (*GENERAL_CONF* register 0x00).
- Set *sd_indirect_control* = 0 (*GENERAL_CONF* register 0x00).

PROCEED WITH:

Action:

- Set *automatic_direct_sdin_switch_off* = 1 once before switching to internal control.
- Continually adapt *VSTART* register 0x25 according to the actual velocity of the TMC4361 – that must be calculated in the µC. Continually adapt *ASTART* according to the actual acceleration (unsigned value) of the TMC4361 – that must be calculated in the µC. Continually set *ASTART*(31) = 0 or 1 according to the current acceleration direction.
- If switching must be prompted, set *sdin_mode* = b'00.

Result:

The internal ramp velocity is started with the value of *VSTART*, and the direction is set automatically on the basis of the external steps that have occurred before. The internal acceleration value is set to:

+*ASTART* if *ASTART*(31) = 0 or

−*ASTART* if *ASTART*(31) = 1.

Read entire documentation; especially the "Supplemental Directives" on page 218.

8. Reference Switches

The reference input signals of the TMC4361 function partly as safety features. The TMC4361 provides a range of reference switch settings that can be configured for many different applications. The TMC4361 offers two hardware switches (STOPL, STOPR) and two additional virtual stop switches (*VIRT_STOP_LEFT*, *VIRT_STOP_RIGHT*). A home reference switch HOME_REF is also available.

Pins used for Reference Switches		
Pin Names	**Type**	**Remarks**
STOPL	Input	Left reference switch.
STOPR	Input	Right reference switch.
HOME_REF	Input	Home switch.
TARGET_REACHED	Output	Reference switch to indicate *XACTUAL=XTARGET.*

Table 22: Pins used for Reference Switches

Dedicated Registers for Reference Switches			
Register Name	**Register Address**		**Remarks**
REFERENCE_CONF	0x01	RW	Configuration of interaction with reference pins.
HOME_SAFETY_MARGIN	0x1E	RW	Region of uncertainty around *X_HOME.*
DSTOP	0x2C	RW	Deceleration value if stop switches STOPL / STOPR or virtual stops are used with soft stop ramps. The deceleration value allows for an automatic linear stop ramp.
POS_COMP	0x32	RW	Free configurable compare position; signed; 32 bits.
VIRT_STOP_LEFT	0x33	RW	Virtual left stop that triggers a stop event at *XACTUAL* ≤ *VIRT_STOP_LEFT*; signed; 32 bits.
VIRT_STOP_RIGHT	0x34	RW	Virtual left stop that triggers a stop event at *XACTUAL* ≥ *VIRT_STOP_RIGHT*; signed; 32 bits.
X_HOME	0x35	RW	Home reference position; signed; 32 bits.
X_LATCH	0x36	RW	Stores *XACTUAL* at different conditions; signed; 32 bits.

Table 23: Dedicated Registers for Reference Switches

Read entire documentation; especially the "Supplemental Directives" on page 218.

8.1. Hardware Switch Support

The TMC4361 offers two hardware switches that can be configured according to your design.

STOPL and STOPR

The hardware provides a left and a right stop in order to stop the drive immediately in case one of them is triggered. Therefore, pin 12 and pin 14 of the motion controller must be used.

NOTE:

→ *Both switches must be enabled before motion occurs.*

In order to enable STOPL correctly, do as follows:

Action:

- Determine the active polarity voltage of STOPL and set *pol_stop_left* (*REFERENCE_CONF* register 0x01) accordingly.
- Set *stop_left_en* =1 (*REFERENCE_CONF* register 0x01).

Result:
The current velocity ramp stops in case the STOPL voltage level matches *pol_stop_left* and *VACTUAL* < 0.

In order to enable STOPR correctly, do as follows:

Action:

- Determine the active polarity voltage of STOPR and set *pol_stop_right* (*REFERENCE_CONF* register 0x01) accordingly.
- Set *stop_right_en* =1 (*REFERENCE_CONF* register 0x01).

Result:
The current velocity ramp stops in case STOPR voltage level matches *pol_stop_right* and *VACTUAL* > 0.

8.1.1. Stop Slope Configuration for Hard or Linear Stop Slopes

The stop slope can be configured for hard or linear stop slopes. Per default, hard stops are selected.

If hard stops are required, do as follows:

OPTION 1: HARD STOP SLOPES

Action:

- Set *soft_stop_en* =0 (*REFERENCE_CONF* register 0x01).

Result:
If one of the stop switches is active and enabled, the velocity ramp is set immediately to *VACTUAL* = 0.

OPTION 2: LINEAR STOP SLOPES

If linear stop ramps are required:

Action:

- Set *soft_stop_en* =1 (*REFERENCE_CONF* register 0x01).

Result:
If one of the stop switches is active and enabled, the velocity ramp is stopped with a linear deceleration slope until *VACTUAL* = 0 is reached. In this case the deceleration factor is determined by *DSTOP*. *VSTOP* is not considered during the stop deceleration slope.

Read entire documentation; especially the "Supplemental Directives" on page 218.

8.1.2. How Active Stops are indicated and reset to Free Motion

When a stop switch becomes active the related status flag is set in the *STATUS* flags register 0x0F. The flag remains active as long as the stop switch remains active.
The particular event is also released in the *EVENTS* register 0x0E, which remains active until the event bit is reset manually. When *VACTUAL* = 0 is reached after the stop event no motion toward this particular direction is possible.

In order to move into the locked direction, the following is required:

PRECONDITION 1:

The particular stop switch is NOT active anymore.

AND/OR

PRECONDITION 2:

The stop switch is disabled (*stop_left/right_en* = 0).

Action:

➢ Set back the active event by reading out the *EVENTS* register 0x0E.

i There is only one exception to this: If an event is selected for the *EVENT_CLEAR_CONF* register in order to inhibit the regular clearing. See information provided in Chapter 5, Page 24.

Result:
The active stop event is reset to free motion into the locked direction.

8.1.3. How to latch Internal Position on Switch Events

It is possible to select four different events to store the current internal position *XACTUAL* in the register *X_LATCH*.

The table below show which transition of the reference signal leads to the *X_LATCH* transfer. For each transition process the specified reference configurations in the *REFERENCE_CONF* register 0x01 must be set accordingly.

Reference Configuration	***pol_stop_left*=0**	***pol_stop_left*=1**	***pol_stop_right*=0**	***pol_stop_right*=1**
latch_x_on_inactive_l=1	STOPL=0 → 1	STOPL=1 → 0	---	---
latch_x_on_active_l=1	STOPL=1 → 0	STOPL=0 → 1	---	---
latch_x_on_inactive_r=1	---	---	STOPR=0 → 1	STOPR = 1→0
latch_x_on_active_r=1	---	---	STOPR=1 → 0	STOPR = 0→1

Table 24: Reference Configuration and Corresponding Transition of particular Reference Switch

Interchange the Reference Switches without Physical Reconnection

If you need to change the directions of the reference switches, do as follows:

Action:

➢ Set *invert_stop_direction*=1 (*REFERENCE_CONF* register 0x01).

Result:
STOPL is now the right reference switch and STOPR is now the left reference switch. Consequently, all configuration parameters for STOPL become valid for STOPR and vice versa.

8.2. Virtual Stop Switches

TMC4361 provides additional virtual limits; which trigger stop slopes in case the specific virtual stop switch microstep position is reached. Virtual stop positions are assigned using the *VIRTUAL_STOP_LEFT* register 0x33 and *VIRTUAL_STOP_RIGHT* register 0x34. In this section, configuration details for virtual stop switches are provided for various design-in purposes.

NOTE:

→ *Virtual stop switches must be enabled in the same manner as nonvirtual reference switches. Hitting a virtual limit switch - by receiving the assigned position - triggers the same process as hitting STOPL or STOPR.*

8.2.1. Enabling Virtual Stop Switches

In order to enable left virtual stop correctly, do as follows:

Action:
- Set *VIRTUAL_STOP_LEFT* register 0x33 according to left stop position.
- Set *virtual_left_limit_en* =1 (*REFERENCE_CONF* register 0x01).

Result:
The actual velocity ramp stops in case *XACTUAL* ≤ *VIRT_STOP_LEFT*. The ramp is stopped according to the selected ramp type.

In order to enable right virtual stop correctly, do as follows:

Action:
- Set *VIRTUAL_STOP_RIGHT* register 0x34 according to right stop position.
- Set *virtual_right_limit_en* =1 (*REFERENCE_CONF* register 0x01).

Result:
The actual velocity ramp stops in case *XACTUAL* ≥ *VIRT_STOP_RIGHT*. The ramp is stopped according to the selected ramp type.

8.2.2. Virtual Stop Slope Configuration

The virtual stop slope can also be configured for hard or linear stop slopes.

If virtual hard stops are required, do as follows:

Action:
- Set *virt_stop_mode* = b'01 (*REFERENCE_CONF* register 0x01).

Result:
If one of the virtual stop switches is active and enabled, the velocity ramp will be set immediately to *VACTUAL* = 0.

If virtual linear stop ramps are required, do as follows:

Action:
- Set *virt_stop_mode* = b'10 (*REFERENCE_CONF* register 0x01).

Result:
If one of the virtual stop switches is active and enabled, the velocity ramp is stopped with a linear deceleration slope until *VACTUAL* = 0 is reached. In this case the deceleration factor is determined by *DSTOP*. *VSTOP* is not considered during the stop deceleration slope.

•→ *Continued on next page.*

8.2.3. How Active Virtual Stops are indicated and reset to Free Motion

At the same time when a virtual stop switch becomes active the related status flag is activated in the *STATUS* flags register 0x0F. The flag remains active as long as the stop switch remains active.

The particular event is also released in the *EVENTS* register 0x0E, which remains active until the event is reset manually. When *VACTUAL* = 0 is reached after the stop event no motion in the particular direction is possible.

In order to move into the locked direction, the following is required:

PRECONDITION 1:

The particular stop switch is NOT active anymore because the actual position does not exceed the specified limit.

AND/OR

PRECONDITION 2:

Virtual stop switch is disabled (*virtual_left/right_limit_en* = 0).

Action:

➢ Set back active event by reading out *EVENTS* register 0x0E.

i There is only one exception to this: If an event is selected for *EVENT_CLEAR_CONF* register in order to inhibit the regular clearing. See information provided in chapter 5, page 24.

Result:

The active virtual stop event bit is reset to free motion into the direction that was locked beforehand.

i *invert_stop_direction* has no influence on *VIRTUAL_STOP_LEFT* and *VIRTUAL_STOP_RIGHT*.

8.3. Home Reference Configuration

In this section home reference switch handling is explained with information about home tracking modes, possible home event configurations and home event monitoring.

Switch Reference Input HOME_REF

For monitoring, the switch reference input HOME_REF is provided.

Perform the following to initiate the homing process:

Action:

- ➢ Assign a ramp according to your needs for the homing process.
- ➢ Enable the home tracking mode with *start_home_tracking* = 1 (*REFERENCE_CONF* register 0x01).
- ➢ Set the correct *home_event* (*REFERENCE_CONF* register 0x01) for the HOME_REF input pin (see *Table* 25, page 59).
- ➢ Start the ramp towards the home switch HOME_REF.

Result:

- When the next home event is recognized by TMC4361, *XACTUAL* is latched to *X_HOME*.
- At the same time, the *start_home_tracking* switch is disabled automatically and
- The *XLATCH_DONE* event is released in the events register 0x0E. This event can be used for an interrupt routine for the homing process in order to avoid polling.

i If an incremental encoder is used to monitor the motion, the N channel can be used to fine-tune the homing position (*home_event* = b'0000). After performing the homing process - as explained before - the N channel events can be used to obtain a more precise home position.

i *X_HOME* can be overwritten manually.

•→ *Continued on next page.*

8.3.1. Home Event Selection

Nine different home events are possible.

i Except for the *home_event* = b'0000, which uses the N signal of an incremental ABN encoder, home events are related to the voltage levels of the HOME_REF input pin:

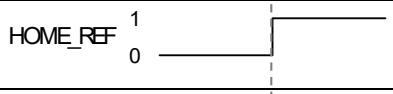
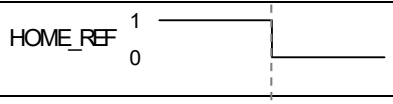
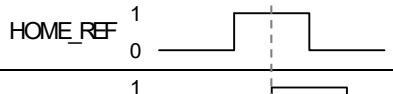
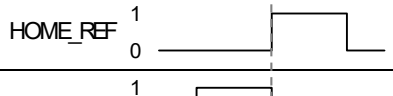
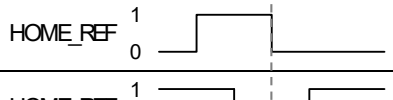
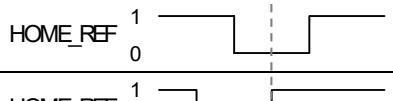
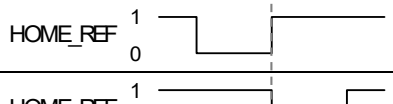
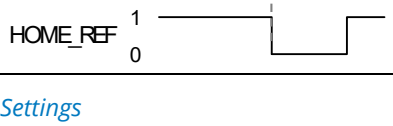
Home Event Selection Table			
home_event	**Description**		**X_HOME (direction: negative / positive)**
b'0011	*HOME_REF* = 0 indicates negative direction in reference to *X_HOME*		
b'1100	*HOME_REF* = 0 indicates positive direction in reference to *X_HOME*		
b'0110	*HOME_REF* = 1 indicates home position	X_HOME in center	
b'0010		X_HOME on the left side	
b'0100		X_HOME on the right side	
b'1001	*HOME_REF* = 0 indicates home position	X_HOME in center	
b'1011		X_HOME on the right side	
b'1101		X_HOME on the left side	

Table 25: Overview of different home_event Settings

8.3.2. HOME_REF Monitoring

An error flag *HOME_ERROR_F* is permanently evaluated. This error flag indicates whether the current voltage level of the HOME_REF reference input is valid in regard to *X_HOME* and the selected home_event.

Defining a Home Range around HOME_REF

In order to avoid false error flags (*HOME_ERROR_F*) because of mechanical inaccuracies, it is possible to setup an uncertainty home range around *X_HOME*. In this range, the error flag is not evaluated.

If you want to define an uncertainty area around *X_HOME*, do as follows:

Action:

➢ Set *HOME_SAFETY_MARGIN* register 0x1E according to the required range in µSteps.

Result:

The homing uncertainties – related to the special application environment – are considered for the ongoing motion. The error flag is NOT evaluated in the following range:

X_HOME − HOME_SAFETY_MARGIN ≤ XACTUAL ≤ X_HOME+HOME_SAFETY_MARGIN

•→ *Continued on next page.*

Continued:
Defining a Home Range around HOME_REF

NOTE:

→ *It is recommended to assign to a higher range value for HOME_SAFETY_MARGIN in which the HOME_REF level is active for the home_events b'0110, b'0010, b'0100, b'1001, b'1011, and b'1101. It avoids false positive HOME_ERROR_Flags.*

→ *After homing with the index channel (home_event = b'0000) for a precise assignment of X_HOME the correct home_event has to be assigned in order to activate the generation of HOME_ERROR_Flags. Note that home_event = b'0000 results in HOME_ERROR_Flag=0 permanently.*

→ *The following examples illustrate the points at which the error flag is release – based on the selected home_event – here for home_event = b'0011 (*), b'1100 (**), b'0110 (***), b'0010 (***), b'0100 (***), b'1001 (****), b'1011 (****), and b'1101 (****).*

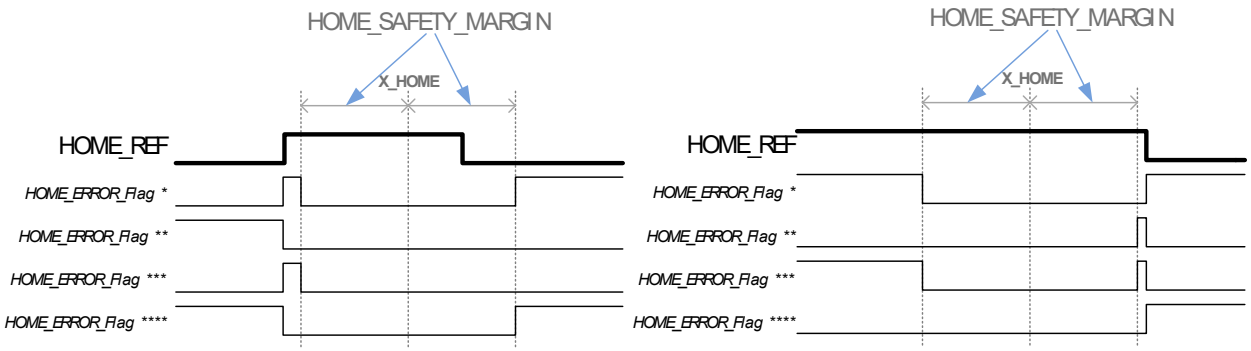


Figure 30: HOME_REF Monitoring and HOME_ERROR_FLAG

8.3.3. Homing with STOPL or STOPR

STOPL and STOPR inputs can also be used as HOME_REF inputs.

OPTION 1: STOPL IS THE HOME SWITCH

Action:

➢ Set *stop_left_is_home* = 1 (*REFERENCE_CONF* register 0x01).

Result:
The stop event at STOPL only occurs when the home range is crossed after STOPL becomes active. The home range is given by *X_HOME* and *HOME_SAFETY_MARGIN*.

OPTION 2: STOPR IS HOME SWITCH

Action:

➢ Set *stop_right_is_home* = 1 (*REFERENCE_CONF* register 0x01).

Result:
The stop event at STOPR only occurs when the home region is crossed after STOPR becomes active. The home region is given by *X_HOME* and *HOME_SAFETY_MARGIN*.

8.4. Target Reached / Position Comparison

In this section, TARGET_REACHED output pin configuration options are explained, as well as different ways how to compare different values internally.

Target Reached Output Pin

TARGET_REACHED output pin forwards the *TARGET_REACHED_F*lag. As soon as *XACTUAL* equals *XTARGET*, TARGET_REACHED is active. Per default, the TARGET_REACHED pin is high active.

To change the TARGET_REACHED output polarity, do the following:

Action:

➢ Set *invert_pol_target_reached* = 1 (bit16 of the *GENERAL_CONF* register 0x00).

Result:
TARGET_REACHED pin is low active.

8.4.1. Connecting several Target-reached Pins

TARGET_REACHED pins can also be configured for a shared signal line in the same way as several INTR pins can configured for one interrupt signal transfer (see section 5.4. (page 27).

To use a Wired-Or or Wired-And behavior, the below described order of action must be executed:

Action:

➢ **Step 1:** Set *intr_tr_pu_pd_en* = 1 (*GENERAL_CONF* register 0x00).

OPTION 1: WIRED-OR

Action:

➢ **Step 2:** Set *tr_as_wired_and* = 0 (*GENERAL_CONF* register 0x00).

Result:
The TARGET_REACHED pin works efficiently as Wired-Or (default configuration).

`i` In case TARGET_REACHED pin is inactive, the pin drive has a weak inactive polarity output. During active state, the output is driven strongly. Consequently, if one of the connected pins is activated, the whole line is set to active polarity.

OPTION 2: WIRED-AND

Action:

➢ **Step 2:** Set *tr_as_wired_and* = 1 (*GENERAL_CONF* register 0x00).

Result:
As long as the target position is not reached, the TARGET_REACHED pin has a strong inactive polarity output. During active state, the pin drive has a weak active polarity output. Consequently, the whole signal line is activated if all connected pins are forwarding the active polarity.

8.4.2. Position Comparison of Internal Values

TMC4361 provides several ways of comparing internal values. The position comparison process is permanently active and associated with one flag and one event.

Basic Comparison Settings

How to compare the internal position with an arbitrary value:

Action:
- Select a comparison value in the *POS_COMP* register 0x32.
- Select *pos_comp_source* = 0 (*REFERENCE_CONF* register 0x01).

Result:
XACTUAL is compared with *POS_COMP*. When *POS_COMP* equals *XACTUAL* the *POS_COMP_REACHED_F*lag becomes set and the *POS_COMP_REACHED* event becomes released.

Select External Position as Comparison Base

How to compare the external position with an arbitrary value:

Action:
- Select a comparison value in the *POS_COMP* register 0x32.
- Select *pos_comp_source* = 1 (*REFERENCE _CONF* register 0x01).

Result:
ENC_POS is compared with *POS_COMP*. When *POS_COMP* equals *ENC_POS* the *POS_COMP_REACHED_F*lag becomes set and the *POS_COMP_REACHED* event becomes released.

NOTE:

→ *Because ENC_POS represents microsteps and not encoder steps, POS_COMP represents also microsteps for the comparison process with external positions.*

→ *In case ENC_POS moves past POS_COMP without assuming the same value as POS_COMP, the POS_COMP_REACHED event is not flagged but is nonetheless listed in the EVENTS register in order to indicate that it has traversed.*

Select Position Compare Flag as TARGET_REACHED Output

A positive comparison result can be forwarded through the INTR pin using the *POS_COMP_REACHED* event as interrupt source.

It is also possible to use the TARGET_REACHED output in order to report the position comparison state (*POS_COMP_REACHED_F*lag) instead of the target reached status.

In order to select the TARGET_REACHED output as *POS_COMP_REACHED_F*lag source, do as follows:

Action:
- Set *pos_comp_output* = b'11 (*REFERENCE _CONF* register 0x01).

Result:
TARGET_REACHED output forwards the *POS_COMP_REACHED_F* status flag.

•→ *Continued on next page.*

Comparison selection grid

In addition to comparing *XACTUAL* / *ENC_POS* with *POS_COMP*, it is also possible to conduct a comparison of one of both parameters with *X_HOME* or *X_LATCH* resp. *ENC_LATCH*. TMC4361 also allows comparison of the revolution counter *REV_CNT against POS_COMP*.

SETTINGS ALERT

Only the selected combination generates the *POS_COMP_REACHED_*Flag and the corresponding event.
Therefore, select *modified_pos_compare* in the *REFERENCE_CONF* register 0x01 as outlined in the table below:

Comparison Selection Grid		
modified_pos_compare \ *pos_comp_source*	**'0'**	**'1'**
'00'	*XACTUAL* vs. *POS_COMP*	*ENC_POS* vs. *POS_COMP*
'01'	*XACTUAL* vs. *X_HOME*	*ENC_POS* vs. *X_HOME*
'10'	*XACTUAL* vs. *X_LATCH*	*ENC_POS* vs. *ENC_LATCH*
'11'	*REV_CNT* vs. *POS_COMP*	

Table 26: Comparison Selection Grid to generate POS_COMP_REACHED_Flag

Read entire documentation; especially the "Supplemental Directives" on page 218.

8.5. Repetitive and Circular Motion

TMC4361 also provides options for auto-repetitive or auto-circular motion. In this section configuration options are explained.

8.5.1. Repetitive Motion to XTARGET

Per default, reaching *XTARGET* in positioning mode finishes a positioning ramp.

In order to continuously repeat the specified ramp, do as follows:

PRECONDITION:

- Set *RAMPMODE*(2) = 1 (positioning mode is active).
- Set up a velocity ramp according to your requirements.

Action:

- Set *clr_pos_at_target* =1 (*REFERENCE_CONF* register 0x01).

Result:

After *XTARGET* is reached (*TARGET_REACHED_F*lag is active), *XACTUAL* is set to 0. As long as *XTARGET* is NOT 0, the ramp restarts in order to reach *XTARGET* again. This leads to repetitious positioning ramps from 0 towards *XTARGET*.

NOTE:

→ *It is possible to change XTARGET during repetitive motion. The reset of XACTUAL to 0 is always executed when XACTUAL equals XTARGET.*

8.5.2. Activating Circular Motion

If circular motion profiles are necessary for your application, TMC4361 offers a position limitation range of *XACTUAL* with an automatic overflow processing. As soon as *XACTUAL* reaches one of the two position range limits (positive / negative), the value of *XACTUAL* is set automatically to the value of the opposite range limit.

In order to activate circular motion, do as follows:

PRECONDITION:

If you want to activate circular motion, *XACTUAL* must be located within the defined range.

PROCEED WITH:

Action:

- Set *X_RANGE* ≠ 0 (register 0x36, only writing access!).
- Set *circular_motion* = 1 (*REFERENCE_CONF* register 0x01).

Result:

The positioning range of *XACTUAL* is limited to: $-X\_RANGE \leq XACTUAL < X\_RANGE$.

When *XACTUAL* reaches the most positive position (*X_RANGE* – 1) and the motion proceeds in positive direction; the next *XACTUAL* value is set to −*X_RANGE*. The same applies to proceeding in negative direction; where (*X_RANGE* −1) is the position after −*X_RANGE*.

i During positioning mode, the motion direction will be dependent on the shortest path to the target position *XTARGET*. For example, if *XACTUAL* = 200, *X_RANGE* = 300 and *XTARGET* = −200, the positioning ramp will find its way across the overflow position (299 → −300) (see Figure A) in *Table* 27 (page 67).

8.5.3. Uneven or Noninteger Microsteps per Revolution

Due to definition of the limitation range, one revolution only consists of an even number of microsteps. TMC4361 provides an option to overcome this limitation.

- Some applications demand different requirements because a revolution consists of an uneven or noninteger number of microsteps.
- TMC4361 allows a high adjustment range of microsteps by using:
  CIRCULAR_DEC register 0x7C.
  This value represents one digit and 31 decimal places as extension for the number of microsteps per one revolution.
- A revolution is completed at overflow position. With every completed revolution the *CIRCULAR_DEC* value is added to an internal accumulation register. In case this register has an overflow, *XACTUAL* remains at its overflow position for one step.
- On average, this leads to the following microsteps per revolution:
  Microsteps/rev = (2 · *X_RANGE*) + *CIRCULAR_DEC* / 2^{31}.

Example 1: Uneven Number of Microsteps per Revolution

One revolution consists of 601 microsteps.
A definition of *X_RANGE* = 300 will only provide:

600 microsteps per revolution (−300 ≤ *XACTUAL* ≤ 299).

Whereas *X_RANGE* = 301 will result in:

602 microsteps per revolution (−301 ≤ *XACTUAL* ≤ 300).

By setting:

CIRCULAR_DEC = 0x80000000 (= 2^{31} / 2^{31} = 1).

An overflow is generated at the decimals accumulation register with every revolution. Therefore, *XACTUAL* prolongs the step at the overflow position for one step every time position overflow is overstepped. This results in a microstep count of 601 per revolution.

Example 2: Noninteger Number of Microsteps per Revolution

One revolution consists of 600.5 microsteps.

By setting:

CIRCULAR_DEC = 0x40000000 (= 2^{30} / 2^{31} = 0.5).

Every second revolution an overflow is produced at the decimals' accumulation register. This leads to a microstep count of 600 every second revolution and 601 for the other half of the revolutions. On average, this leads to 600.5 microsteps per revolution.

Example 3: Noninteger and uneven Number of Microsteps per Revolution

One revolution consists of 601.25 microsteps.

By setting:

CIRCULAR_DEC = 0xA0000000 (= (2^{31} + 2^{29}) / 2^{31} = 1.25).

With every revolution an overflow is produced at the decimals' accumulation register. Furthermore, at every fourth revolution an additional overflow occurs, which leads to another prolonged step. This leads to a microstep count of 601 for three of four revolutions and 602 for every fourth revolution. On average, this results in 601.25 microsteps per revolution.

Read entire documentation; especially the "Supplemental Directives" on page 218.

8.5.4. Release of the Revolution Counter

By overstepping the position overflow, the internal *REV_CNT* register is increased by one revolution as soon as *XACTUAL* oversteps from (*X_RANGE* –1) to *–X_RANGE* or is decreased by one revolution as soon as *XACTUAL* oversteps in the opposite direction.

The information about the number of revolutions can be obtained by reading out register 0x36, which by default is the *X_LATCH* register (read only).

In order to gain information on the number of revolutions:

Action:

➢ Set *circular_cnt_as_xlatch* = 1 (*GENERAL_CONF* register 0x00).

Result:

Register 0x36 cease to display the *X_LATCH* value. Instead, the revolution counter *REV_CNT* can be read out at this register address.

NOTE:

→ *As soon as circular motion is inactive (*circular_motion = *0),* REV_CNT *is reset to 0.*

8.6. Blocking Zones

8.6.1. Activating Blocking Zones during Circular Motion

During circular motion, virtual stops can be used to set blocking zones. Positions inside these blocking zones are NOT dedicated for motion.

In order to activate the blocking zone, do as follows:

PRECONDITION:

Circular motion is activated (*circular_motion* = 0) and properly assigned (*X_RANGE* ≠ 0).

PROCEED WITH:

Action:

➢ Set *VIRTUAL_STOP_LEFT* register 0x33 as left limit for the blocking zone.

➢ Set *VIRTUAL_STOP_RIGHT* register 0x34 as right limit for the blocking zone.

➢ Enable both virtual limits as explained in section 8.2.1 (page 56).

Result:

The blocking zone reaches from *VIRTUAL_STOP_LEFT* to *VIRTUAL_STOP_RIGHT*. During positioning, the path from *XACTUAL* to *XTARGET* does not lead through the blocking zone; which can result in a longer path compared to the direct path through the blocking zone (see Figure B1 in *Table* 27 (page 67).

However, the selected virtual stop deceleration ramp is initiated as soon as one of the limits is reached. This can result from the velocity mode or if the target *XTARGET* is located in the blocking zone.

•→ *Continued on next page!*

Blocking Zone Definition

The following positions are located within the blocking zone:

XACTUAL ≤ *VIRT_STOP_LEFT*

AND / OR

XACTUAL ≥ *VIRT_STOP_RIGHT*

NOTE:

→ *In case* VIRTUAL_STOP_LEFT < VIRTUAL_STOP_RIGHT, *one of these conditions must be met in order to be located inside the blocking zone.*

→ *In case* VIRTUAL_STOP_LEFT > VIRTUAL_STOP_RIGHT, *both conditions must be met in order to be located inside the blocking zone.*

8.6.2. Circular Motion with and without Blocking Zone

The table below shows circular motion (*X_RANGE* = 300). The green arrow depicts the path which is chosen for positioning.
The shortest path selection is shown in Figure A and the consideration of blocking zones are shown in Figures B1 and B2.

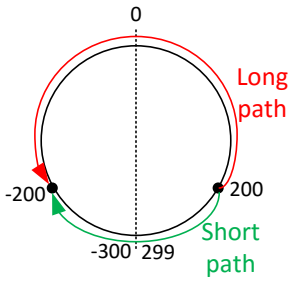
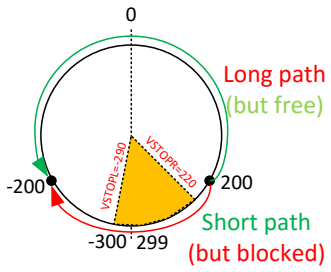
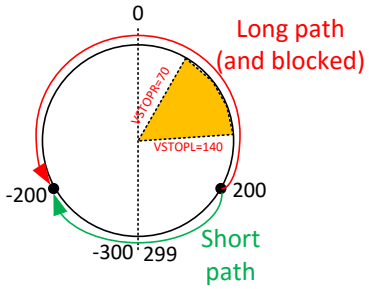
Circular Motion with (B1, B2) and Without (A) Blocking Zone		
A	**B1**	**B2**
		

Table 27: Circular motion (X_RANGE = 300)

Moving out of the Blocking Zone

When *XACTUAL* is located inside the blocking zone, it is possible to move out without redefining the blocking zone.

In order to get out of the blocking zone, do the following:

Action:

- Activate positioning mode: *RAMPMODE*(2) = 1.
- Configure velocity ramp according to your needs.
- Clear virtual stop events by reading out *EVENTS* register 0x0E.
- Set regular target position *XTARGET* outside of the blocking zone.

Result:
TMC4361 initiates a ramp with the shortest way to the target *XTARGET*.

`i` In order to match an incremental encoder in the same manner, select *circular_enc_en* =1 (*REFERENCE_CONF* register 0x01).

9. Ramp Timing and Synchronization

TMC4361 provides various options to initiate a new ramp. By default, every external register change is assigned immediately to the internal registers via an SPI input. With a proper start configuration, ramp sequences can be programmed without any intervention in between.

Synchronization Opportunities

Three levels of ramp start complexity are available. Predefined ramp starts are available, which are independent of SPI data transfer that are explained in the subsequent section 9.1. (page 69).

Two optional features can be configured that can either be used individually or combined, which are as follows:

Shadow Register Set

A complete shadow motion register set can be loaded into the actual motion registers in order to start the next ramp with an altered motion profile.

Target Position Pipeline

Different target positions can be predefined, which are then activated successively. This pipeline can be configured as cyclic; and/or it can also be utilized to sequence different parameters.

Masterless synchronization

Also, another start state “busy” can be assigned in order to synchronize several motion controllers for one single start event without a master.

Dedicated Ramp Timing Pins		
Pin Names	**Type**	**Remarks**
START	Input and Output	External start input to get a start signal or external start output to indicate an internal start event.

Table 28: Dedicated Ramp Timing Pins

Dedicated Ramp Timing Registers			
Register Name	**Register Address**		**Remarks**
START_CONF	0x02	RW	The configuration register of the synchronization unit.
START_OUT_ADD	0x11	RW	Additional active output length of external start signal.
START_DELAY	0x13	RW	Delay time between start triggers and start signal.
X_PIPE0… 7	0x38…0x3F	RW	Target positions pipeline and/or parameter pipeline.

Table 29: Dedicated Ramp Timing Registers

9.1. Basic Synchronization Settings

Usually, a ramp can be initiated internally or externally. Note that a start trigger is not the start signal itself but the transition slope to the active start state. After a defined delay, the internal start signal is generated.

9.1.1. Start Signal Trigger Selection

For ramp start configuration, consider the following steps:

Action:

- ➢ Choose internal or external start trigger(s).
- ➢ Set the triggers according to the table below.

i All triggers can be used separately or in combination.

Start Trigger Configuration Table	
***trigger_events* = *START_CONF*(8:5)**	**Result**
b'0000	No start signal will be generated or processed further.
b'xxx0	Set *trigger_events*(0) = 0 for internal start triggers only. The internally generated start signal is forwarded to the **START pin** that is assigned as **output**.
b'xxx1	Set *trigger_events*(0) = 1 for an external start trigger. The **START pin** is assigned as **input**. For START input take filter settings into consideration. See chapter 4, page 20.
b'xx1x	TARGET_REACHED event is assigned as start signal trigger for the ramp timer.
b'x1xx	VELOCITY_REACHED event is assigned as start signal trigger for the ramp timer.
b'1xxx	POSCOMP_REACHED event is assigned as start signal trigger for the ramp timer.

Table 30: Start Trigger Configuration

9.1.2. User-specified Impact Configuration of Timing Procedure

Per default, every SPI datagram is processed immediately. By selecting one of the following enable switches, the assignment of SPI requests to registers *XTARGET*, *VMAX*, *RAMP_MODE,* and *GEAR_RATIO* is uncoupled from the SPI transfer. The value assignment is only processed after an internally generated start signal.

In order to influence the impact of the start signal on internal parameter assignments, do the following:

Action:

- ➢ Choose between the following options as shown in the table below.

Start Enable Switch Configuration Table *(All switches can be used separately or in combination.)*	
start_en* =START_CONF*(4:0)	**Result**
b'xxxx1	*XTARGET* is altered only after an internally generated start signal.
b'xxx1x	*VMAX* is altered only after an internally generated start signal.
b'xx1xx	*RAMPMODE* is altered only after an internally generated start signal.
b'x1xxx	*GEAR_RATIO* is altered only after an internally generated start signal.
b'1xxxx	Shadow register is assigned as active ramp parameters after an internally generated start signal. This is explained in more detail in section 9.2. (page 74).

Table 31: Start Enable Switch Configuration

9.1.3. Delay Definition between Trigger and internally generated Start Signal

Per default, the trigger is closely followed by the internal start signal.

In order to delay the generation of the internal start signal, do the following:

Action:
➢ Set *START_DELAY* register 0x13 according to your specification.

Result:
When a start trigger is recognized, the internal start signal is generated after *START_DELAY* clock cycles.

Prioritizing External Input

Per default, an external trigger is also delayed for the internal start signal generation.

In order to immediately prompt an external start, trigger to an internally generated start signal (regardless of a defined delay), do the following:

Action:
➢ Set *immediate_start_in* = 1 (*START_CONF* register 0x02).

Result:
When an external start trigger is recognized, the internal start signal is generated immediately, even if the internal start triggers have already initiated a timing process with an active delay.

START Pin Polarity

The START pin can be used either as input or as output pin. However, the active voltage level polarity of the START pin can be selected with one configuration switch in the *START_CONF* register 0x02.

Per default, the voltage level transition from high to low triggers a start signal (START is an input), or START output indicates an active START event by switching from high to low level.

In order to invert active START polarity, do as follows:

Action:
➢ Set *pol_start_signal* = 1 (*START_CONF* register 0x02).

Result:
The START pin is high active. The voltage level transition from low to high triggers a start signal (START is an input), or START output indicates an active START event by switching from low to high level.

9.1.4. Active START Pin Output Configuration

Per default, the active output voltage level of the START pin lasts one clock cycle.

In order to extend this time span, do the following:

Condition:
➢ START pin is assigned as output: *trigger_events*(0) = 1.

Action:
➢ Set *START_OUT_ADD* register 0x11 according to your specification.

Result:
The active voltage level lasts (*START_OUT_ADD* + 1) clock cycles.

Read entire documentation; especially the "Supplemental Directives" on page 218.

9.1.5. Ramp Timing Examples

The following three examples depict SPI datagrams, internal and external signal levels, corresponding velocity ramps, and additional explanations. SPI data is transferred internally at the end of each datagram.

Ramp Timing Example 1

In this example, the velocity value change is executed immediately.

Process Description

- The new *XTARGET* value is assigned after TARGET_REACHED has been set and *START_DELAY* has elapsed.
- A new ramp does not start at the end of the second ramp because no new *XTARGET* value is assigned.
- START is an output.
- Internal start signal forwards with a step length of (*START_OUT_ADD* + 1) clock cycles.

This is how external devices can be synchronized:

Parameter Settings Timing Example 1	
Parameter	**Setting**
RAMPMODE	b'101
start_en	b'00001
trigger_events	b'0010
START_DELAY	>0
START_OUT_ADD	>0
pol_start_signal	1

Table 32: Parameter Settings Timing Example 1

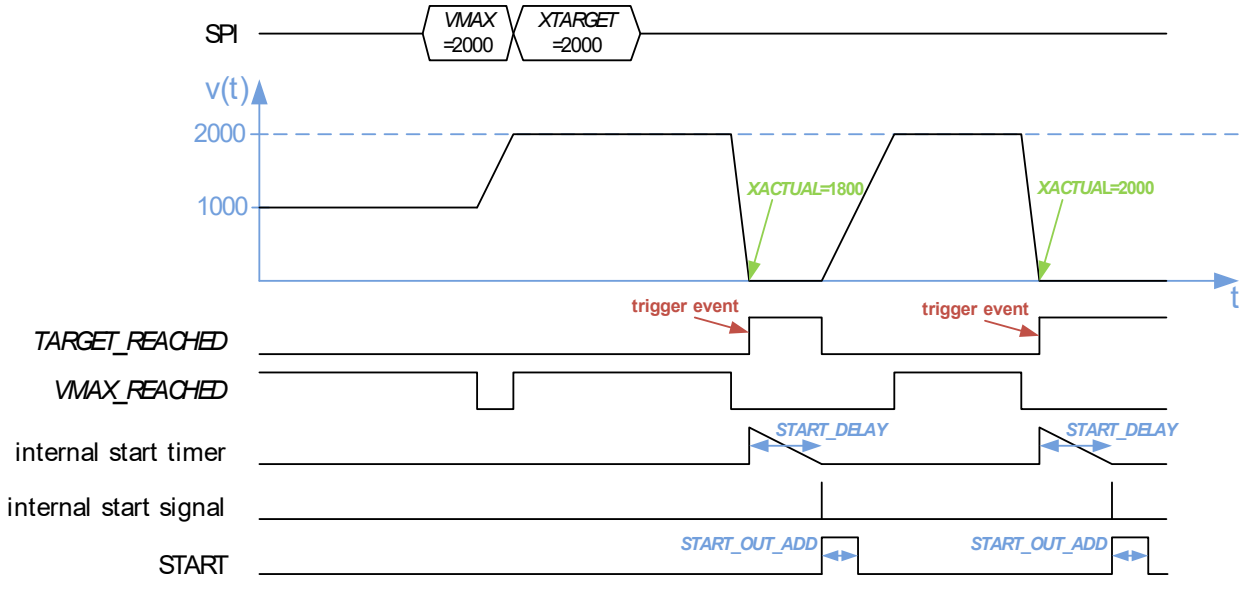


Figure 31: Ramp Timing Example 1

Read entire documentation; especially the "Supplemental Directives" on page 218.

Ramp Timing Example 2

In this example, the velocity value and the ramp mode value change is executed after the first start signal.

Process Description

- The new ramp mode becomes positioning mode with S-shaped ramps.
- The ramp then stops at target position *XTARGET* because of the ramp mode change.
- A further *XTARGET* change starts the ramp again.
- The ramp is initiated as soon as the start delay is completed, which was triggered by the first *TARGET_REACHED* event.
- The active START output signal lasts only one clock cycle.

Parameter Settings Timing Example 2	
Parameter	**Setting**
RAMPMODE	b'001 → b'110
start_en	b'00111
trigger_events	b'0110
START_DELAY	>0
START_OUT_ADD	0
pol_start_signal	0

Table 33: Parameter Settings Timing Example 2

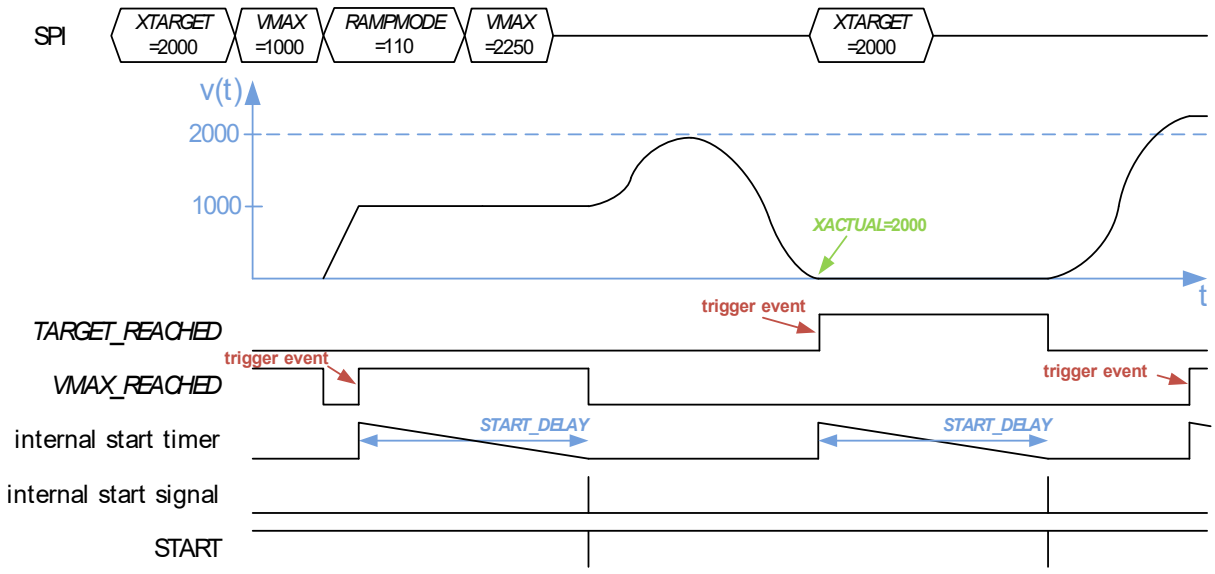


Figure 32: Ramp Timing Example 2

Ramp Timing Example 3

Process Description

In this example external start signal triggers are prioritized by making use of *START_DELAY* > 0 and simultaneously setting *immediate_start_in* to 1.

- When *XACTUAL* equals *POSCOMP* the start timer is activated and the external start signal in between is ignored.
- The second start event is triggered by an external start signal. The *POSCOMP_REACHED* event is ignored.

  The third start timer process is disrupted by the external START signal, which is forced to be executed immediately due to the setting of: *immediate_start_in* = 1.

Parameter Settings Timing Example 3	
Parameter	**Setting**
RAMPMODE	b'000
start_en	b'00010
trigger_events	b'1001
immediate_start_in	0 → 1
START_DELAY	>0
pol_start_signal	1

Table 34: Parameter Settings Timing Example 3

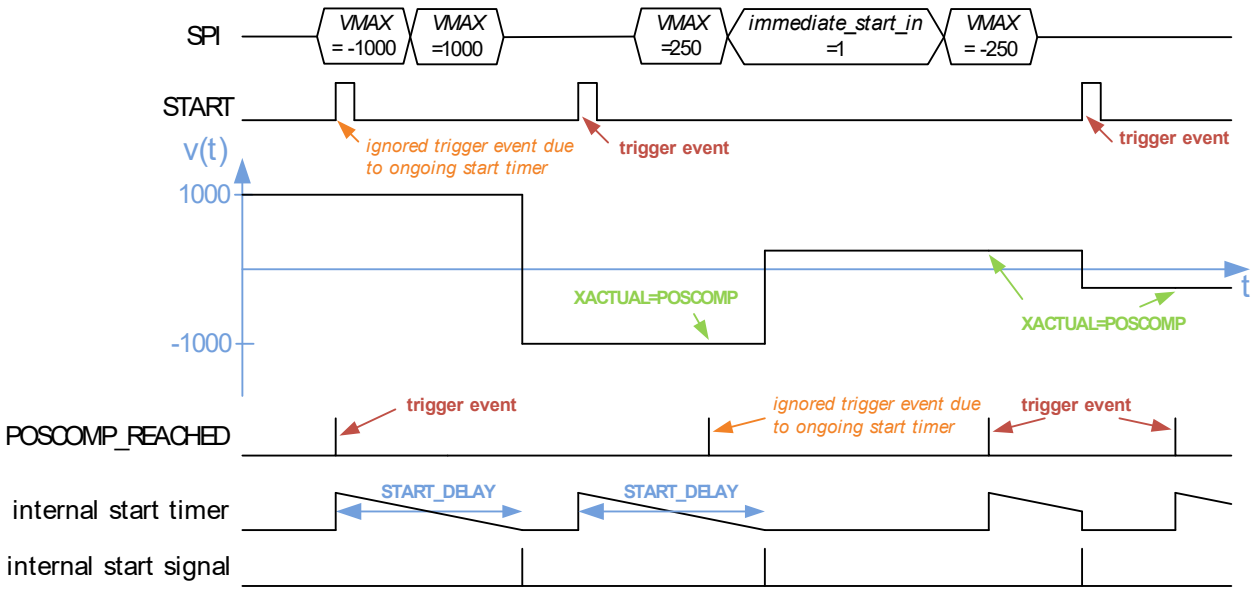


Figure 33: Ramp Timing Example 3

9.2. Shadow Register Settings

Some applications require a complete new ramp parameter set for a specific ramp situation / point in time. TMC4361 provides up to 14 shadow registers, which are loaded into the corresponding ramp parameter registers after an internal start signal is generated.

Enabling Shadow Registers

In order to enable shadow registers, do as follows:

Action

- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02), see section 9.1.2 (page 69).

Result:
With every successive internal start signal the shadow registers are loaded into the corresponding active ramp register.

Enabling Cyclic Shadow Registers

It is also possible to write back the current motion profile into the shadow motion registers to swap ramp motion profiles continually.

In order to enable cyclic shadow registers, do as follows:

Action

- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02) , see section 9.1.2 (page 69).
- Set *cyclic_shadow_regs* = 1 (*START_CONF* register 0x02).

Result:
With every successive internal start signal the shadow registers are loaded into the corresponding active ramp register, whereas the active motion profile is loaded into the shadow registers.

•→ *Continued on next page.*

9.2.1. Shadow Register Configuration Options

Four different optional shadow register assignments are available to match the shadow register set according to your selected ramp type. The available options are described on the next pages.

i Please note that the only difference between the configuration of shadow option 3 and 4 is that *VSTART* is exchanged by *VSTOP* for the transfer of the shadow registers.

Option 1: Shadow Default Configuration

If the whole ramp register is needed to set in a single level stack, do as follows:

Action:

- Set *shadow_option* = b'00 (*START_CONF* register 0x02).
- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02)

Action:

- **Default configuration:** Set *cyclic_shadow_regs* = 0 (*START_CONF* register 0x02)
- **Optional configuration:** Set *cyclic_shadow_regs* = 1 (*START_CONF* register 0x02)

Result:

Every relevant motion parameter is altered at the next internal start signal by the corresponding shadow register parameter. In case cyclic shadow registers are used, the shadow register set is altered by the current motion profile set.

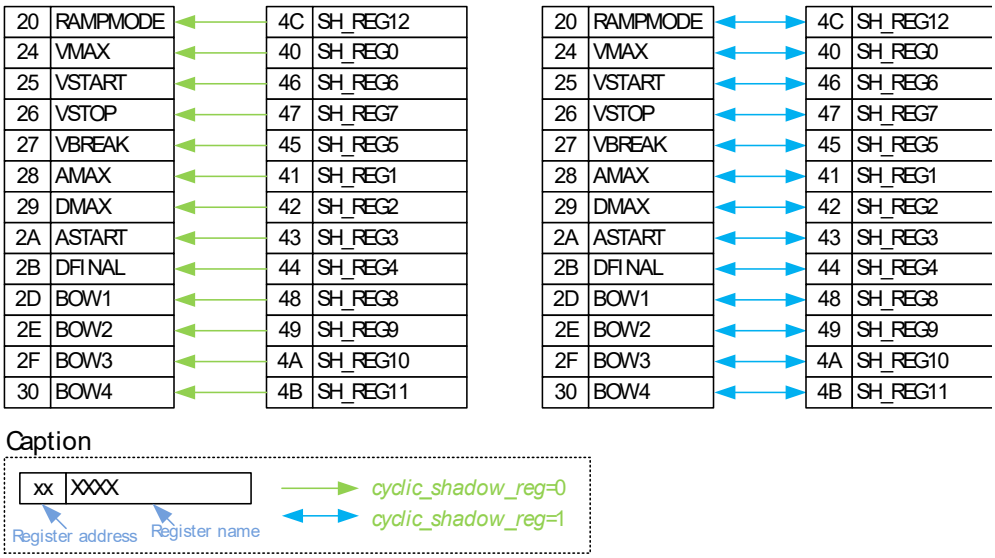


Figure 34: Single-level Shadow Register Option to replace complete Ramp Motion Profile.

i Green arrows show default settings

i Blue arrows show optional settings.

•→ Continued on next page.

Option 2: Double-stage Shadow Register Set for S-shaped Ramps

In case S-shaped ramps are configured, a double-stage shadow register set can be used. Seven relevant motion parameters for S-shaped ramps are affected when the shadow registers become active.

In order to use a double-stage shadow register pipeline for S-shaped ramps, do as follows:

Action:

- Set *shadow_option* = b'01 (*START_CONF* register 0x02).
- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02).

Action:

- **Default configuration:** Set *cyclic_shadow_regs* = 0 (*START_CONF* register 0x02).
- **Optional configuration:** Set *cyclic_shadow_regs* =1 (*START_CONF* register 0x02)

Result:

Seven motion parameters (*VMAX*, *AMAX*, *DMAX*, *BOW1…4*) are altered at the next internal start signal by the corresponding shadow register parameters (*SH_REG0…6*). Simultaneously, these shadow registers are exchanged with the parameters of the second shadow stage (*SH_REG7…13*).

In case cyclic shadow registers are used, the second shadow register set (*SH_REG7…13*) is altered by the current motion profile set, e.g. 0x28 (*AMAX*) is written back to 0x48 (*SH_REG8*).

The other ramp registers remain unaltered.

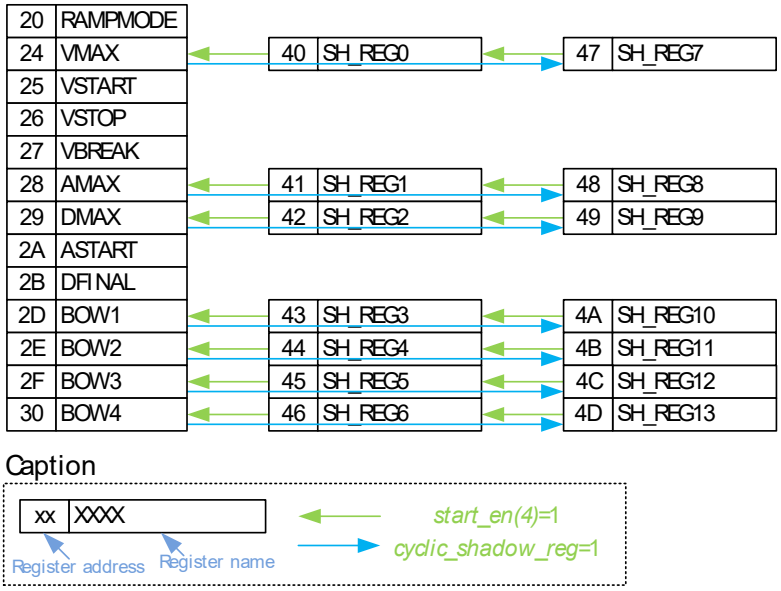


Figure 35: Double-stage Shadow Register Option 1, suitable for S-shaped Ramps.

i Green arrows show default settings

i Blue arrows show optional settings.

•→ Description is continued on next page.

Shadow Option3 Double-stage Shadow Register Set for Trapezoidal Ramps (*VSTART*)

In case trapezoidal ramps are configured, a double-stage shadow register set can be used. Seven relevant motion parameters for trapezoidal ramps are affected when the shadow registers become active.

In order to use a double-stage shadow register pipeline for trapezoidal ramps, do as follows:

Action:

- Set *shadow_option* = b'10 (*START_CONF* register 0x02).
- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02)

Action:

- **Default configuration:** Set *cyclic_shadow_regs* = 0 (*START_CONF* register 0x02).
- **Optional configuration:** Set *cyclic_shadow_regs* = 1 (*START_CONF* register 0x02).

Result:

Seven motion parameters (*VMAX*, *AMAX*, *DMAX, ASTART, DFINAL, VBREAK*, and *VSTART*) are altered at the next internal start signal by the corresponding shadow register parameters (*SH_REG0…6*). Simultaneously, these shadow registers are exchanged with the parameters of the second shadow stage (*SH_REG7…13*).
If cyclic shadow registers are used, the second shadow register set (*SH_REG7…13*) is altered by the current motion profile set, e.g. 0x27 (*VBREAK*) is written back to 0x4C (*SH_REG12*). The other ramp registers remain unaltered.

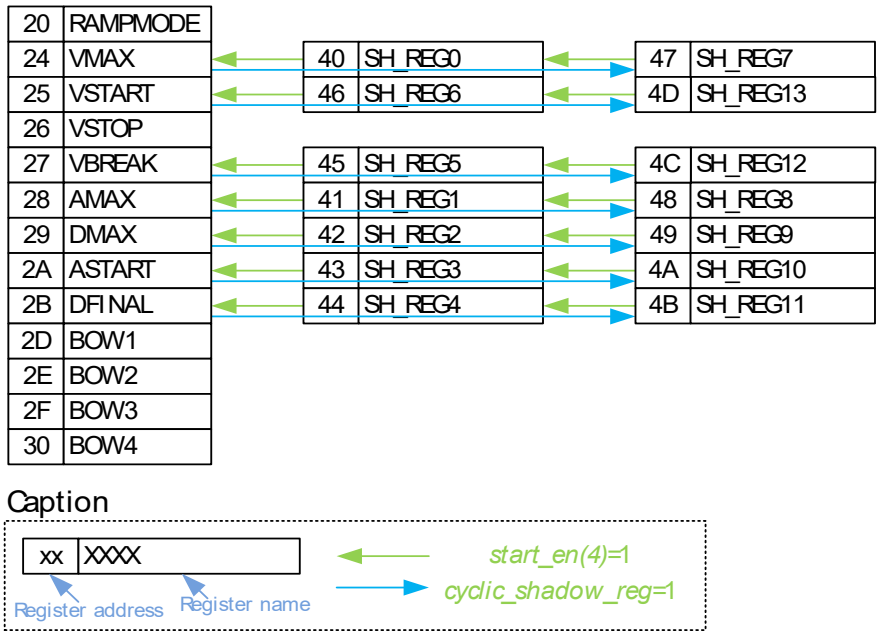


Figure 36: Double-stage Shadow Register Option 2, suitable for Trapezoidal Ramps.

i Green arrows show default settings.

i Blue arrows show optional settings.

•→ Description is continued on next page.

Shadow Option4: Double-stage Shadow Register Set for Trapezoidal Ramps (*VSTOP*)

In case trapezoidal ramps are configured, a double-stage shadow register set can be used. Seven relevant motion parameters for trapezoidal ramps are affected when the shadow registers become active.

In order to use a double-stage shadow register pipeline for trapezoidal ramps, do as follows:

Action:

- Set *shadow_option* = b'10 (*START_CONF* register 0x02).
- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02)

Action:

- **Default configuration:** Set *cyclic_shadow_regs* = 0 (*START_CONF* register 0x02).
- **Optional configuration:** Set *cyclic_shadow_regs* = 1 (*START_CONF* register 0x02)

Result:

Seven motion parameters (*VMAX*, *AMAX*, *DMAX, ASTART, DFINAL, VBREAK*, and *VSTOP*) are altered at the next internal start signal by the corresponding shadow register parameters (*SH_REG0…6*). Simultaneously, these shadow registers are exchanged with the parameters of the second shadow stage (*SH_REG7…13*).
If cyclic shadow registers are used, the second shadow register set (*SH_REG7…13*) is altered by the current motion profile set, e.g. 0x26 (*VSTOP*) is written back to 0x4D (*SH_REG13*). The other ramp registers remain unaltered.

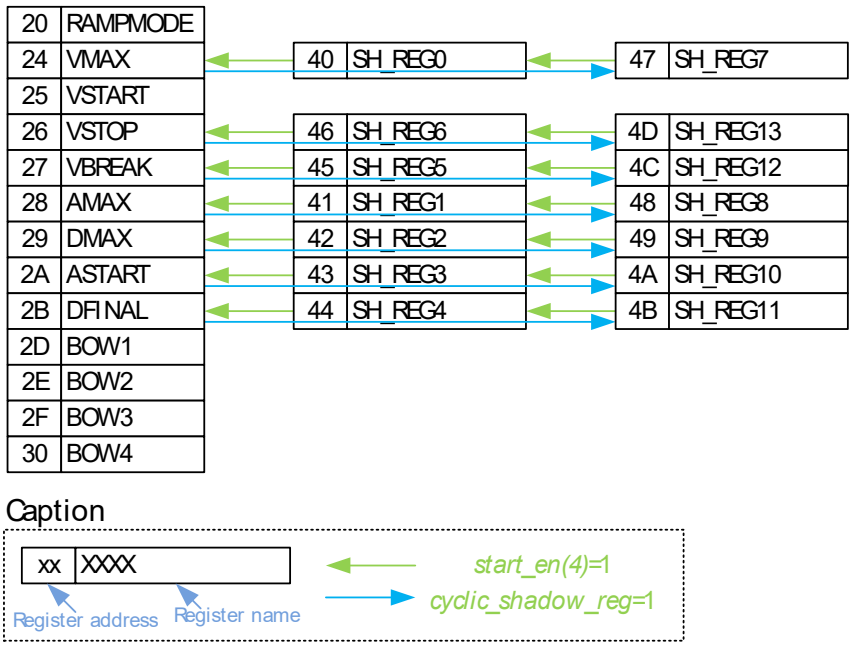


Figure 37: Double-Stage Shadow Register Option 3, suitable for Trapezoidal Ramps

i Green arrows show default settings.

i Blue Arrows show optional settings.

•→ *Turn page to see **Areas of Special Concern** pertaining to this section.*

AREAS OF SPECIAL CONCERN

The values of ramp parameters, which are not selected by one of the four shadow options stay as originally configured, until the register is changed through an SPI write request.
Also, the last stage of the shadow register pipeline retains the values until they are overwritten by an SPI write request if no cyclic shadow registers are selected.

9.2.2. Delayed Shadow Transfer

Up to 15 internal start signals can be skipped before the shadow register transfer is executed.

In order to skip a defined number of internal start signals for the shadow transfer, do as follows:

Action:

- Set *shadow_option* according to your specification.
- Set *start_en*(4) = 1 and select one or more *trigger_events* (*START_CONF* register 0x02)
- ***OPTIONAL CONFIGURATION:*** Set *cyclic_shadow_regs* = 1.
- Set *SHADOW_MISS_CNT* ≠ 0 (*START_CONF* register 0x02) according to the number of consecutive internal start signals that you specify to be ignored.

Result:
The shadow register transfer is not executed with every internal start signal. Instead, the specified number of start signals is ignored until the shadow transfer is executed through the (*SHADOW_MISS_CNT*+1)th start signal.

The following figure shows an example of how to make use of *SHADOW_MISS_CNT*, in which the shadow register transfer is illustrated by an internal signal sh_reg_transfer. The signal miss counter *CURRENT_MISS_CNT* can be read out at register address *START_CONF* (23:20):

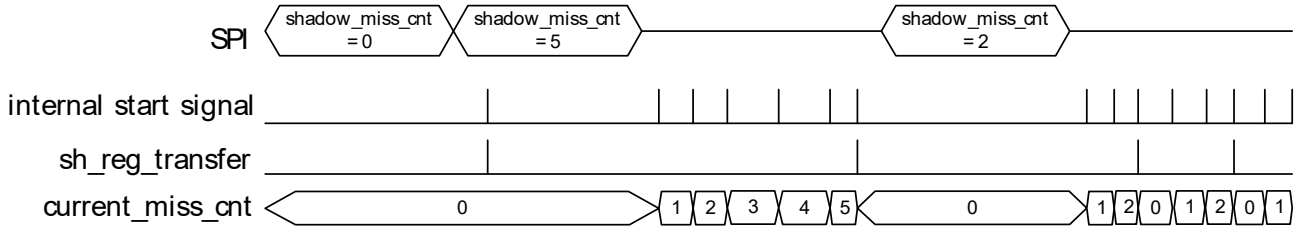


Figure 38: SHADOW_MISS_CNT Parameter for several internal Start Signals

•→ Description is continued on next page.

AREAS OF SPECIAL CONCERN

Internal calculations to transfer the requested shadow BOW values into internal structures require at most (320 / f_{CLK}) [sec]. before any shadow register transfer is prompted, it is necessary to wait for the completion of all internal calculations for the shadow bow parameters.

In order to make this better understood the following example is provided for a double-stage shadow pipeline for S-shaped ramps:

PRECONDITION:

Shadow register transfer is activated (*start_en*(1) = 1 and one or more *trigger_events* are selected) for S-shaped ramps (*shadow_option* = b'01)

Action

- Set *SH_REG0*, *SH_REG1*, *SH_REG2* (shadow register for *VMAX*, *AMAX*, *DMAX*).
- Set *SH_REG3*, *SH_REG4*, *SH_REG5*, *SH_REG6* (shadow register for *BOW1*...*4*).
- Ensure that no shadow register transfer occurs during the next 320 / f_{CLK} [s].

Result:
Shadow register transfer can be initiated after this time span.

AREAS OF SPECIAL CONCERN

It is strongly recommended that the values of the shadow ramp parameters are only transferred during standstill of the current ramp (*VACTUAL* = 0); especially if the *RAMP_MODE* is changed.

In case the transfer is prompted during motion, *VACTUAL* must be constant for all ramp types. If S-Shaped ramps are selected, please ensure that *VACTUAL* is configured to remain constant for a definite delay $t_{SHADOW\_TRANSFER}$ before you assign any new *VMAX* or *XTARGET* values, because a new internal recalculation process is triggered:

$$t_{SHADOW\_TRANSFER} = x = \frac{\sqrt{\max(BOW3[pps^3], BOW4[pps^3]) \cdot \mathrm{VMAX[pps]}}}{56 \cdot BOW3[pps^3]}$$

If positioning mode is selected, *XTARGET* must be set accordingly in order to maintain a constant *VACTUAL* value.

It is also required that *VMAX* and its shadow counterpart must be equal to avoid alteration of *VACTUAL* during $t_{SHADOW\_TRANSFER}$.

Read entire documentation; especially the "Supplemental Directives" on page 218.